

CloudEmu: A Zero-Dependency, In-Memory Multi-Cloud Service Emulation Library for Go

Nitin Kumar, Gursewak Singh, Devansh Arora

Department of Computer Science, Chitkara University,
Rajpura, Punjab, India
nitin2001.be22@chitkara.edu.in,
devansh1490.be22@chitkara.edu.in,
gursewak1601.be22@chitkara.edu.in

ABSTRACT

Cloud services are hard to test against. They are slow, they cost money, and they break in unpredictable ways. LocalStack needs Docker. Moto is Python-only and AWS-only. Official emulators cover one service from one provider each. We wrote CloudEmu, a Go library that fakes 10 cloud services for AWS, Azure, and GCP entirely in memory, with no external dependencies at all. Three layers keep things clean: portable APIs on top, driver interfaces in the middle, provider backends at the bottom. We went beyond basic CRUD. CloudEmu handles eight behaviors that real clouds exhibit: it generates metrics when VMs start or stop, evaluates alarm thresholds, checks IAM policies with wildcard matching, deduplicates FIFO messages over five-minute windows, compares database values numerically instead of lexicographically, routes poison messages to dead-letter queues, tracks per-operation costs, and fires serverless functions when queue messages arrive. Our test suite has 31 tests. They all pass in under 500ms with zero flakiness.

Keywords—cloud testing, service emulation, test doubles, multi-cloud, in-memory fakes, Go

I. INTRODUCTION

Pick any production system and count the cloud services it depends on. Object storage, VMs, a database or two, some message queues, DNS, monitoring, a load balancer, IAM roles, maybe a Lambda function. Now multiply that by two or three providers if the company runs multi-cloud. Testing all of those interactions without actually calling the cloud? That is the problem.

You can test against the real cloud, but that means network round-trips, a growing AWS bill, and flaky tests caused by shared state or rate limits. LocalStack [1] gets rid of the bill, but now you need Docker running in CI, startup takes seconds, and you only get AWS. Moto [2] works great if your stack is Python and AWS. If you write Go, it does nothing for you. The official emulators — Azurite, DynamoDB Local, the Firebase suite — each handle one service from one cloud. Mix and match at your own risk.

None of these tools solve the multi-cloud problem. More and more teams run workloads across two or three providers for resilience, cost savings, or compliance reasons. Testing that kind of code today means stitching together separate emulators for each cloud — if compatible emulators even exist.

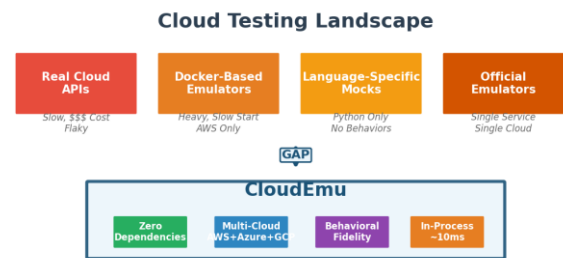


Fig. 1. Cloud testing landscape and the gap CloudEmu fills.

We boiled the gap down to four requirements that, as far as we could find, no single tool meets today:

- **Zero external dependencies** — no Docker, no network, no JVM, no separate processes.
- **Multi-cloud coverage** — AWS, Azure, and GCP services from a single library.
- **Behavioral fidelity** — realistic cloud semantics: alarm evaluation, IAM enforcement, deduplication.
- **In-process, instant-start** — suitable for unit tests that must run in milliseconds.

CloudEmu is our answer to that gap. It is an open-source Go library delivering in-memory emulations of 10 services per provider (30 total), organized in a three-layer architecture that keeps portable APIs separate from driver interfaces and provider-specific backends.

In short, this paper contributes:

- **A three-layer architecture** for multi-cloud service emulation with clean separation of concerns.
- **Eight realistic cloud behaviors:** auto-metrics, alarm evaluation, IAM policies, FIFO dedup, numeric comparisons, dead-letter queues, cost simulation, and serverless triggers.

- **A practical evaluation** with 31 tests covering all services and cross-service interactions.

II. BACKGROUND

A. The Cloud Testing Problem

Cloud-native applications talk to managed services through provider SDKs. A 2019 systematic review [3] surveyed 147 primary studies across six areas and flagged the lack of lightweight, portable testing tools as a stubborn gap. More recently, Kumar [4] reported that teams with continuous performance testing in CI/CD catch 76% of issues before deployment, compared to 31% with periodic manual testing — a strong incentive to make cloud tests fast and easy to run.

B. Test Double Taxonomy

Meszaros [5] splits test doubles into five categories: Dummy, Stub, Spy, Mock, and Fake. As Fowler [6] pointed out, Fakes sit at the top of that ladder — they are the hardest to build but the most useful. CloudEmu falls squarely in the Fake camp: its emulations hold state, enforce constraints, evaluate policies, and fire cross-service side effects. That puts it well beyond Stubs (canned answers) or Mocks (call-verification wrappers).

C. The Multi-Cloud Gap

Every existing tool picks a lane and stays in it. LocalStack [1] and Moto [2] are AWS-only. Azurite [7] does Azure Storage, period. Firebase Emulator Suite [8] covers GCP and nothing else. Go CDK [9], Libcloud [10], and CloudBridge [11] try to abstract across providers, but they wrap real SDKs underneath — so you still need credentials, network access, and a running cloud account.

III. RELATED WORK

A. Cloud Infrastructure Simulators

CloudSim [12], CloudSim Plus [13], and CloudSim 7G [14] solve a different problem entirely. They simulate data-center hardware — hosts, VMs, network links — for scheduling and resource-allocation research. What they do not do is expose service-level APIs like S3 or DynamoDB. An application developer cannot call `CreateBucket()` on CloudSim.

B. Cloud Service Emulators

LocalStack [1] is the closest relative — it covers 70+ AWS services with high fidelity, but it requires Docker and only supports AWS. Moto [2] intercepts Python’s boto3 calls and is Python-only. Official emulators (Azurite [7], DynamoDB

Local, Firebase Suite [8]) each cover one service from one provider.

C. Cloud Abstraction Libraries

Go CDK [9] offers portable Go APIs across providers, but under the hood it still calls the real SDKs — you need live credentials. CloudBridge [11], jclouds [15], and Libcloud [10] take the same approach in Python and Java.

D. Service Virtualization

Service virtualization tools like WireMock [16], Hoverfly, and Mountebank work at the HTTP level — they record and replay request/response pairs. That is useful for regression testing but cannot enforce business logic like IAM policies, alarm thresholds, or deduplication windows.

E. Testing Methodology

Fowler [6] and Meszaros [5] established the test-double taxonomy that the industry still uses. Wang et al. [17] found that 66% of Apache projects use mocking, but almost all rely on Stubs or Mocks rather than Fakes. Building a Fake is expensive; CloudEmu amortizes that cost so individual teams do not have to bear it.

Service Coverage Comparison

CloudEmu AWS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
CloudEmu Azure	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
CloudEmu GCP	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
LocalStack	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Moto	✓	✓	✓	✓	✗	✗	✓	✓	✗	✓
Azurite	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
Firebase Emulator	✗	✗	✓	✓	✗	✗	✓	✗	✗	✗
	Storage	Compute	Database	Serverless	Network	Monitor	IAM	DBS	LI	ML

Fig. 9. Service coverage heatmap.

IV. ARCHITECTURE AND DESIGN

A. Design Goals

- **Zero dependencies** — the library compiles and runs with nothing beyond Go’s standard library. No Docker, no JVM, no pip install.
- **Behavioral fidelity** — we wanted the fakes to act like the real thing: state machines that reject illegal transitions, policies that actually get evaluated, alarms that actually fire.
- **Thread safety** — every operation is safe for concurrent use. We guard shared state with `sync.RWMutex` throughout.
- **Deterministic testing** — a pluggable FakeClock replaces wall-clock time so tests produce identical results on every run.

- **Uniform coverage** — AWS, Azure, and GCP each get the same ten services. No provider is a second-class citizen.

B. Three-Layer Architecture

Go’s database/sql package splits interface from implementation cleanly. We took that same idea and added a third layer (Table I).

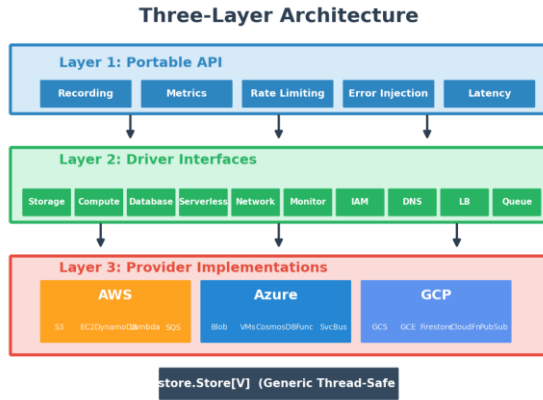


Fig. 2. Three-layer architecture with provider implementations.

TABLE I. THREE-LAYER ARCHITECTURE

Layer	Location	Purpose
Portable API	storage/, compute/, ...	Recording, metrics, rate limiting
Driver Interfaces	*/driver/driver.go	Go interfaces per service
Provider Impls	providers/{aws,azure,gcp}/	In-memory memstore backends

C. Service Coverage

All three providers cover the same ten service categories (Table II), so switching a test from AWS to GCP is mostly a matter of changing the factory call.

TABLE II. SERVICE COVERAGE ACROSS THREE CLOUD PROVIDERS

Category	AWS	Azure	GCP
Storage	S3	Blob Storage	GCS
Compute	EC2	VMs	GCE
Database	DynamoDB	CosmosDB	Firestore
Serverless	Lambda	Functions	CloudFn
Network	VPC	VNet	VPC
Monitoring	CloudWatch	Monitor	Monitoring
IAM	IAM	Azure IAM	Cloud IAM
DNS	Route 53	Azure DNS	Cloud DNS
Load Bal.	ELB	Azure LB	Cloud LB
Msg Queue	SQS	Service Bus	Pub/Sub

D. Cross-Service Wiring

In production clouds, services talk to each other behind the scenes. Launch an EC2 instance and CloudWatch starts collecting metrics automatically. CloudEmu mirrors this: each provider factory injects the monitoring service into the compute service through SetMonitoring(). The payoff is emergent behavior — spinning up a VM can trip a

CloudWatch alarm, just like it would in a real AWS account.

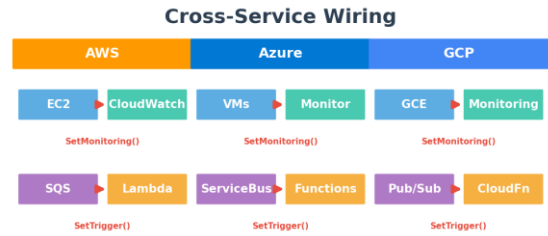


Fig. 3. Cross-service wiring across providers.

E. Deterministic Time

Anything that depends on wall-clock time goes through a Clock interface instead of calling time.Now() directly. In tests, we swap in a FakeClock so we can jump forward by exact durations — useful for checking deduplication window boundaries and alarm evaluation periods without waiting around.

V. REALISTIC CLOUD BEHAVIORS

CloudEmu ships with eight behaviors designed to match how the real clouds actually work. Each one is implemented identically across all three providers.

A. Auto-Metric Generation on Compute Lifecycle

Launch a VM and five metrics show up in the monitoring mock. Stop it and those metrics drop to zero. Start, reboot, terminate — each one pushes fresh datapoints. A test can create an alarm, launch an instance, and confirm the alarm fires, all without manually injecting any metric data. Worth noting: GCP expresses CPU as a fraction (0.25) while AWS and Azure use a percentage (25.0).

TABLE III. METRIC EMISSION PER LIFECYCLE OP

Operation	CPU	Net	Disk	Pts
Run	25.0	1024/512	100/50	5
Start	25.0	1024/512	100/50	1
Stop	0.0	0.0/0.0	0.0/0.0	1
Reboot	25.0	1024/512	100/50	1
Terminate	0.0	0.0/0.0	0.0/0.0	1

B. Alarm Auto-Evaluation

Every time new metric data comes in, the monitoring mock checks whether any alarm cares about that namespace and metric name. If one does, it pulls datapoints from inside the evaluation window (Period times EvaluationPeriods), runs the requested statistic — average, sum, min, max, or sample count — and compares the result to the alarm’s threshold. If the comparison flips, the alarm state changes. One subtle detail: we drop the write lock before running this evaluation,

otherwise the alarm check would deadlock trying to read metric data while the write lock is still held.

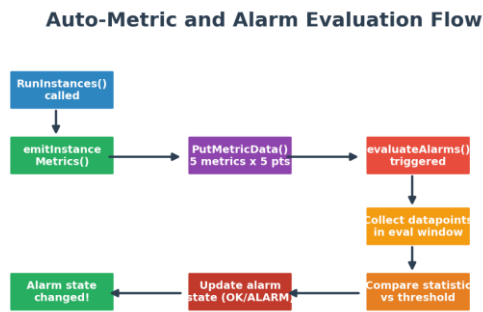


Fig. 6. Auto-metric generation and alarm evaluation flow.

C. IAM Policy Evaluation

When you call `CheckPermission(principal, action, resource)`, the mock gathers every policy attached to that principal — both user-level and role-level. It parses the JSON in each policy document, then tries to match the requested action and resource against the statements using `wildcardMatch()`, which supports `*` and `?` patterns. The logic follows real IAM rules: if any statement explicitly says Deny, that wins immediately. Otherwise, there has to be at least one Allow. No match at all means denied by default.

D. FIFO Message Deduplication

FIFO queues keep a deduplication index keyed by `DeduplicationID`. If the same ID shows up within five minutes, the queue hands back the original `MessageID` without creating a duplicate. After the five-minute window expires, the ID is treated as new. `FakeClock` lets us test the exact boundary of that window without sleeping.

E. Numeric-Aware Database Comparisons

Database scan filters call a `compareValues()` helper that tries to parse both sides as floating-point numbers. When both parse successfully it does a numeric comparison; otherwise it falls back to string ordering. The practical effect: 9 sorts before 10, instead of after it (the way a naive lexicographic comparison would order them).

F. Dead-Letter Queues

Each message queue mock lets you attach a dead-letter queue (DLQ) at creation time via `DeadLetterConfig`, which takes a `TargetQueueURL` and a `MaxReceiveCount`. Once a message has been received more than `MaxReceiveCount` times, the next `ReceiveMessages` call silently moves it to the DLQ — mirroring what SQS, Service Bus, and Pub/Sub do with poison messages in production.

G. Cost Simulation

A `cost.Tracker` keeps a running tally of estimated spend per operation. It comes preloaded with default per-operation rates for all ten service categories and exposes `TotalCost()`, `CostByService()`, `CostByOperation()`, and `Reset()`. You can also plug in custom rates with `SetRate()`. The idea is to let CI pipelines fail when a test scenario would blow a budget threshold — a sanity check you cannot get from mocks that ignore pricing.

H. Serverless Triggers

`SetTrigger()` connects a queue to a serverless function. After that, every message sent to the queue automatically calls the function with the message payload — replicating how SQS triggers Lambda, Service Bus triggers Azure Functions, and Pub/Sub triggers Cloud Functions in production. Call `RemoveTrigger()` to disconnect them.

VI. IMPLEMENTATION

A. Technology Choices

CloudEmu compiles under Go 1.25. It is one module (github.com/stackshy/cloudemu) with zero imports outside the standard library. Go made sense here because of goroutines, `sync.RWMutex` for cheap concurrency safety, generics to avoid type assertions in the store layer, and compilation fast enough that the full build barely registers in CI.

Implementation Components



Fig. 4. Internal implementation components.

B. Generic In-Memory Store

Every mock stores its data in `memstore.Store[V]` — basically a `map[string]V` behind a `sync.RWMutex`. It has four methods: `Get`, `Set`, `Delete`, `All`. The generic parameter `V` means each service works with its own struct directly. No `interface{}` casting, no runtime type checks.

C. State Machine for Compute Lifecycle

VM lifecycle transitions run through a shared finite state machine: `pending` → `running` → `stopping` → `stopped` → `pending (restart)`, `running` → `shutting-down` → `terminated` (terminal state), and `running` → `restarting` → `running`.

(reboot). Any request that would violate these transitions gets a FailedPrecondition error.

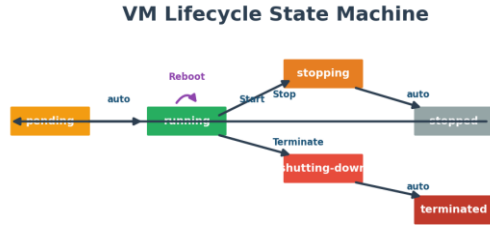


Fig. 5. VM lifecycle state machine.

D. ID Generation

AWS: arn:aws:<service>:::<resource>

Azure:

/subscriptions/<sub>/resourceGroups/<rg>/providers/<type>
>/<resource>

GCP: projects/<project>/<collection>/<resource>

E. Cross-Cutting Concerns

A Recorder that logs every API call with its timestamp, so tests can assert on call sequences. A Metrics Collector tracking call counts and durations per operation — handy for spotting unexpected call patterns. Rate Limiter — uses a token bucket. When tokens run out, the next call gets a Throttled error. Good for checking whether retry logic actually works. Error Injector — tell it to fail on the 3rd call (or whichever), and it will. Handy when you need to test a code path that only runs on transient failures.

F. Code Size

As of now the codebase sits at around 13,850 lines of Go across 78 files, not counting tests. Tests add another 1,600 lines on top of that.

VII. EVALUATION

A. Correctness

31 tests organized in three tiers:

Tier 1: Service-Level (8 tests). These cover storage lifecycle (create/read/delete with pagination), compute state-machine transitions, database CRUD with queries, and a cross-provider portability check that runs the same scenario on all three clouds.

Tier 2: Cross-Cutting (4 tests). These verify the four cross-cutting wrappers: forced error injection, token-bucket rate limiting, API call recording, and per-operation metrics collection.

Tier 3: Realistic Behaviors (19 tests).

TABLE IV. REALISTIC BEHAVIOR TEST SUITE

Test	Behavior	Prov.
AutoMetricGen	VM emits 5×5 metrics	All 3
AlarmAutoEval	Threshold fires alarm	All 3
AlarmByMetrics	VM→metrics→alarm	All 3
StopEmitsZero	Stop zeros, alarm	All 3
StartEmitsRun	Start running vals	All 3
IAMPermission	Policy+wildcards	All 3
FIFODedup	5-min dedup	All 3
NumericCmp	Numeric DB cmp	All 3
ScanOps	≤,≥,BEGINS WITH	All 3
DeadLetterQ	Poison routing	All 3
CostTracker	Op cost tally	All
ServerlessTrig	Queue→function	All 3

All 31 tests pass deterministically with zero flakiness — a direct benefit of in-memory state and fake clocks eliminating timing-related non-determinism.

B. Integration Testing

In addition to unit tests, we wrote 6 HTTP integration tests inside a companion application (ZipZopCloud) that wraps CloudEmu behind REST handlers. These tests exercise the library through actual HTTP round-trips, verifying that behavior holds when accessed through a real server stack rather than direct Go calls.

C. Performance

Operations run in microseconds — they are just map lookups behind a mutex. No container boots, no port binding, no network stack. The full 31-test suite clocks in under half a second on a laptop.

D. Comparison with Existing Tools

TABLE V. COMPARISON WITH EXISTING TOOLS

	Ours	LStack	Moto	Azurite	FBEm
Lang	Go	Py	Py	Node	Node
Cloud	3	1	1	1	1
Docker	No	Yes	No	No	No
Start	0ms	3s	500ms	2s	4s
InProc	Yes	No	Yes	No	No
Fidel.	High	High	Med	Med	Med
Svcs	30	70+	~50	3	5
Deps	0	2	1	1	2

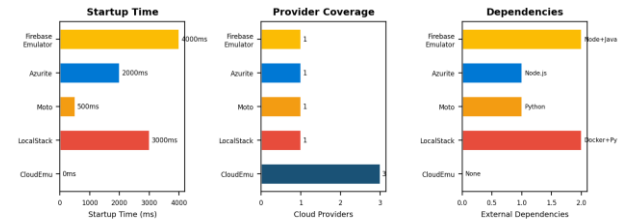


Fig. 7. Comparison of CloudEmu with existing cloud testing tools.

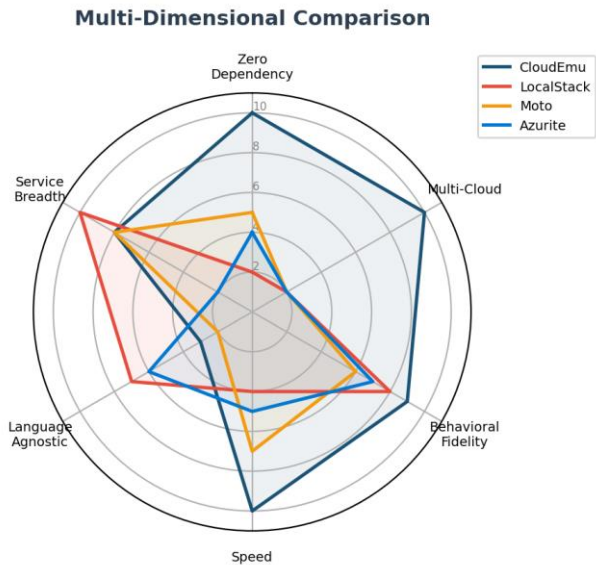


Fig. 8. Radar comparison across key dimensions.

VIII. DISCUSSION AND LIMITATIONS

A. Limitations

Service depth is shallow in places. The S3 mock, for instance, skips versioning, lifecycle policies, and cross-region replication. We chose breadth (10 services $\times$ 3 providers) over depth in this first release.

There is no network layer. CloudEmu cannot simulate partitions, DNS resolution failures, or TLS handshake problems — everything runs in-process via direct Go calls.

The library is Go-only right now. Python, Java, or TypeScript teams cannot use it today, though an HTTP wrapper (see Future Work) would open it up.

IAM emulation is approximate. We support action/resource matching with wildcards, but condition keys, permission boundaries, and service control policies are not yet implemented.

B. Threats to Validity

Our correctness claims depend on how accurately we read the official cloud docs. We did not run conformance tests against live AWS, Azure, or GCP endpoints, so subtle behavioral gaps are possible — particularly for undocumented edge cases or region-specific quirks. The 31-test suite covers the behaviors we built, but it is not an exhaustive specification of any real cloud service.

C. Future Work

- Go deeper on individual services — S3 versioning, Lambda cold-start simulation, and richer database query operators top the list.

- Wrap CloudEmu in an HTTP server so Python, Java, and TypeScript projects can use it too, the same way LocalStack exposes AWS-compatible endpoints.
- Build a conformance suite that runs the same assertions against both CloudEmu and real cloud APIs, catching drift automatically.
- Publish proper benchmarks measuring speedup over Docker-based emulators, especially in CI/CD where container startup time dominates.

IX. CONCLUSION

We built CloudEmu to scratch a specific itch: testing multi-cloud Go code without Docker, without live credentials, and without waiting. The library delivers in-memory emulations of 10 services each for AWS, Azure, and GCP, organized in three clean layers. Eight behaviors push these emulations past simple CRUD stubs: automatic metric generation on compute lifecycle events, threshold-based alarm evaluation, IAM policy matching with wildcard support, FIFO message deduplication, numeric-aware database comparisons, dead-letter queue routing, per-operation cost simulation, and serverless event-source triggers.

As far as we know, no existing tool combines multi-cloud coverage, in-process execution, zero dependencies, and behavioral fidelity in a single package. Our 31 tests run in under 500ms with zero startup overhead and zero flakiness. CloudEmu is open-source at github.com/stackshy/cloudemu, and we hope it saves other teams the same pain it saved us.

REFERENCES

- [1] W. Hummer et al., "LocalStack: A fully functional local cloud stack," GitHub, 2024.
- [2] S. Gamaat et al., "Moto: Mock AWS Services," GitHub, 2024.
- [3] P. Garousi et al., "A systematic review on cloud testing," *ACM Comput. Surv.*, vol. 52, no. 4, 2019.
- [4] R. S. Kumar, "Cloud-native performance testing in CI/CD pipelines," ResearchGate, 2024.
- [5] G. Meszaros, *xUnit Test Patterns*. Addison-Wesley, 2007.
- [6] M. Fowler, "TestDouble," *martinfowler.com*, 2006.
- [7] Microsoft, "Azurite open-source emulator," MS Learn, 2024.
- [8] Google, "Firebase Local Emulator Suite," Firebase docs, 2024.
- [9] Google, "Go Cloud Development Kit," GitHub, 2024.
- [10] Apache, "Apache Libcloud," 2024.
- [11] N. Goonasekera et al., "CloudBridge," *Proc. XSEDE16, ACM*, 2016.
- [12] R. N. Calheiros et al., "CloudSim," *Softw. Pract. Exper.*, vol. 41, no. 1, pp. 23–50, 2011.
- [13] M. C. Silva Filho et al., "CloudSim Plus," *Proc. IFIP/IEEE IM*, pp. 400–406, 2017.
- [14] R. Buyya et al., "CloudSim 7G," *Softw. Pract. Exper.*, 2025.
- [15] Apache, "Apache jclouds," 2024.
- [16] T. Roome, "WireMock," *wiremock.io*, 2024.
- [17] H. Wang et al., "Mocking frameworks in Apache," *Empir. Softw. Eng.*, vol. 28, no. 6, 2023.

APPENDIX: CROSS-PROVIDER METRIC MAPPING

TABLE VI. CROSS-PROVIDER METRIC MAPPING

Metric	AWS	Azure	GCP
CPU	CPUUtil=25	%CPU=25	cpu/util=0.25
NetIn	NetIn=1024	NetInTot=1024	recv_bytes=1024
NetOut	NetOut=512	NetOutTot=512	sent_bytes=512
DiskRd	DiskRdOps=100	DiskRd/s=100	rd_ops=100
DiskWr	DiskWrOps=50	DiskWr/s=50	wr_ops=50